

WhatsApp Chat Analyzer - Interview Preparation

1. The Elevator Pitch (High-Level Overview)

"I built a web application that takes an exported WhatsApp chat text file, parses the unstructured text data into a structured format using Regular Expressions, and provides a dashboard of data visualizations. It allows users to view overall group statistics or drill down into individual user behavior, analyzing metrics like message frequency, active hours, common vocabulary, and emoji usage."

2. Tech Stack

- * Frontend/UI: Streamlit (for rapid prototyping of data web apps)
- * Data Manipulation: Pandas (for structuring and grouping data)
- * Visualizations: Matplotlib & Seaborn
- * NLP & Text Processing: Regular Expressions (re), WordCloud, urlextract, emoji module

3. Architecture & Code Structure

Your project follows a clean, modular architecture split into three main files:

- preprocessing.py (The Data Pipeline): Responsible for taking raw, unstructured .txt data and converting it into a clean Pandas DataFrame. It extracts features like dates, times, days, and user names.
- helper.py (The Engine): Contains all the analytical logic and functions. It filters data, calculates statistics, and prepares dataframes/objects specifically for visualizations.
- chapter4.py (The Frontend): The main Streamlit entry point. It handles file uploads, user inputs (sidebar selections), and renders the final charts and UI elements.

4. Key Technical Highlights

A. Unstructured Data Parsing using Regex

- * The Problem: WhatsApp exports chats as raw text.
- * The Solution: You used the `re` module in preprocessing.py to split the string based on the date-time pattern `r"\d{1,2}/\d{1,2}/\d{2,4},\s\d{1,2}:\d{1,2}s\-\s"`. This allowed you to cleanly separate the timestamps from the actual message content.

B. Handling "Hinglish" and Custom Stop Words

- * The Problem: Standard NLP libraries have stop words for English. However, WhatsApp chats often contain slang or mixed languages (Hinglish, like "hai", "bhi", "ki").
- * The Solution: You used a custom stop_hinglish.txt file to iterate through the text and remove any words found in this custom dictionary.

C. Feature Engineering for Time-Series Analysis

- * You engineered several new columns in Pandas: year, month, day_name, hour, and period.
- * You specifically calculated a period column which allowed you to create the intricate Seaborn Activity Heatmap showing the busiest times of day over the week.

D. Handling Group Notifications vs. Real Messages

- * You wrote logic to distinguish between actual messages (User: Message) and WhatsApp system notifications. If a message doesn't split properly by a colon, you categorize the user as group_notification.

5. Possible Interview Questions & Answers

Q: How does your application handle large chat files? Will it crash?

A: Currently, the app loads the file directly into memory using Streamlit's `getvalue()` and processes it with Pandas. Since WhatsApp text files are generally just a few megabytes at most, Pandas handles it very efficiently in memory.

Q: Why did you separate your code into helper.py and preprocessing.py instead of writing it all in one file?

A: Separation of concerns. I wanted to keep the Streamlit UI code clean and readable. By moving the

complex regex parsing to preprocessing.py and the Pandas grouping logic to helper.py, the code becomes much easier to test, debug, and maintain.

Q: How did you calculate the "Most Busy Users" percentage?

A: In helper.py, I used the Pandas `value_counts()` function on the 'user' column, divided it by the total shape of the dataframe, and multiplied by 100 to get the exact percentage breakdown.

Q: How do you handle emojis?

A: I used the Python emoji library. I iterate through the message strings and use the `is_emoji()` function to extract them into a list, then use the Python Counter module to find the most frequently used ones.